

FPP3 Chapter 3.7 Exercises (1, 3, 5, 7, 9)

2176071 KimSooMin

2026-04-13

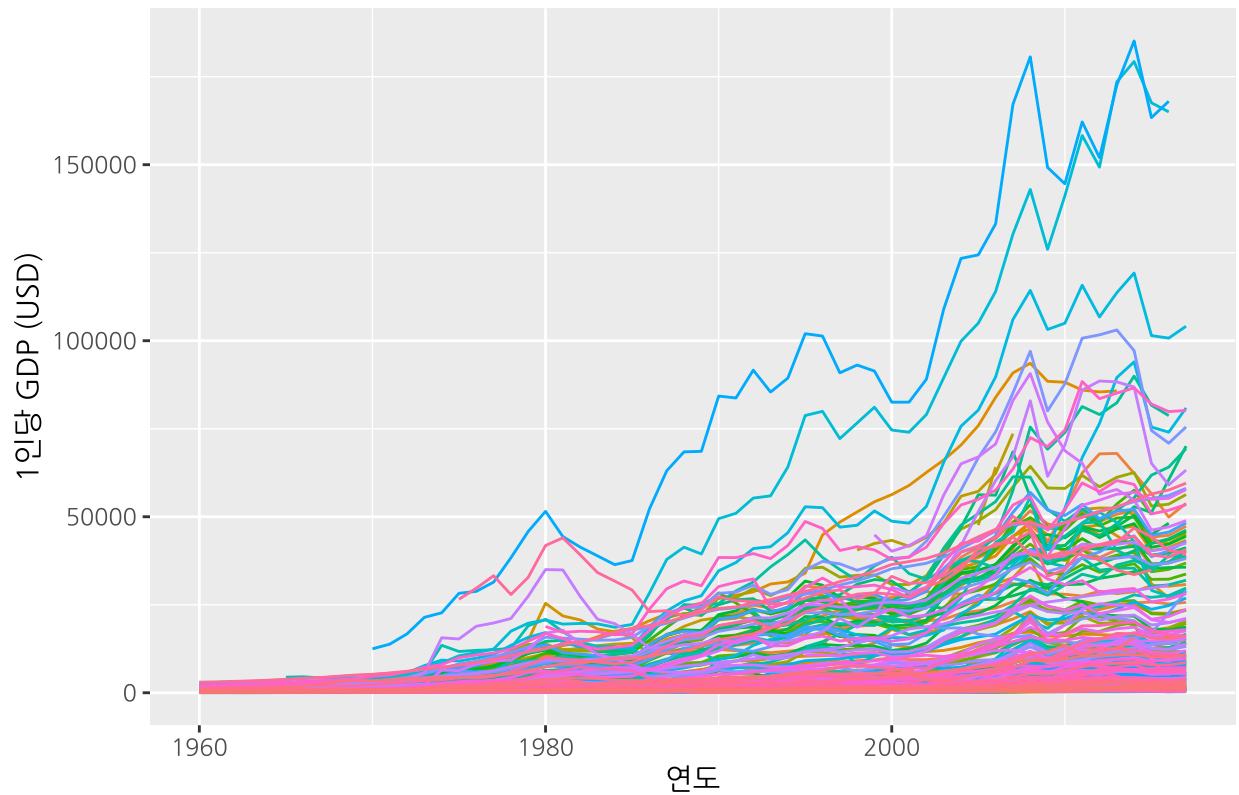
Exercise 1

문제: Consider the GDP information in `global_economy`. Plot the GDP per capita for each country over time. Which country has the highest GDP per capita? How has this changed over time?

(1) 국가별 1인당 GDP 시계열 플롯

```
global_economy |>
  mutate(GDP_per_capita = GDP / Population) |>
  autoplot(GDP_per_capita) +
  labs(
    title = "1 GDP",
    y = "1 GDP (USD)",
    x = " "
  ) +
  theme(legend.position = "none")
```

국가별 1인당 GDP



(2) 가장 높은 1인당 GDP를 가진 국가

```
global_economy |>
  as_tibble() |>
  mutate(GDP_per_capita = GDP / Population) |>
  filter(!is.na(GDP_per_capita)) |>
  group_by(Country) |>
  summarise(max_gdp_pc = max(GDP_per_capita, na.rm = TRUE)) |>
  arrange(desc(max_gdp_pc)) |>
  head(10)
```

```
## # A tibble: 10 x 2
##   Country      max_gdp_pc
##   <fct>      <dbl>
## 1 Monaco      185153.
## 2 Liechtenstein 179308.
## 3 Luxembourg   119225.
## 4 Norway       103059.
## 5 Macao SAR, China 94004.
## 6 Bermuda      93606.
## 7 San Marino    90683.
## 8 Isle of Man   89942.
## 9 Qatar         88565.
## 10 Switzerland  88416.
```

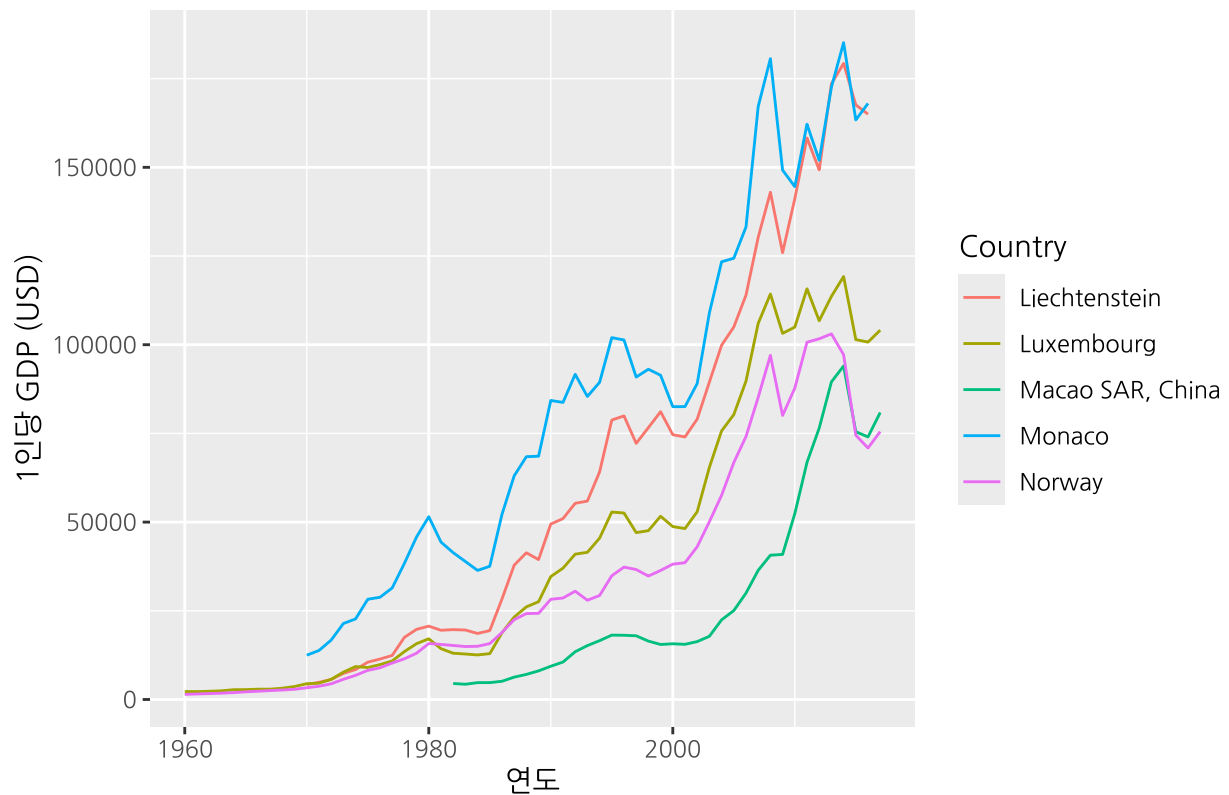
```

top_countries <- global_economy |>
  as_tibble() |>
  mutate(GDP_per_capita = GDP / Population) |>
  filter(!is.na(GDP_per_capita)) |>
  group_by(Country) |>
  summarise(max_gdp_pc = max(GDP_per_capita, na.rm = TRUE)) |>
  arrange(desc(max_gdp_pc)) |>
  head(5) |>
  pull(Country)

global_economy |>
  mutate(GDP_per_capita = GDP / Population) |>
  filter(Country %in% top_countries) |>
  autoplot(GDP_per_capita) +
  labs(
    title = " 5 1 GDP ",
    y = "1 GDP (USD)",
    x = " ",
    colour = " "
  )

```

상위 5개국 1인당 GDP 추세



(3) 해석

위 표를 보면 Monaco가 가장 높은 1인당 GDP를 기록하고 있으며, 그 뒤를 Liechtenstein, Luxembourg 등 소규모 고소득 국가들이 따르고 있습니다.

시간의 흐름에 따라 상위권 국가들의 1인당 GDP는 모두 상승하는 추세를 보이지만, 국가별로 성장 속도에 뚜렷한 차이가 있습니다. Monaco와 Liechtenstein은 특히 2000년대 이후 가파른 상승세를 보이며 다른 국가들과의 격차를 벌였습니다. Luxembourg는 꾸준하고 안정적인 성장 패턴을 유지하고 있습니다.

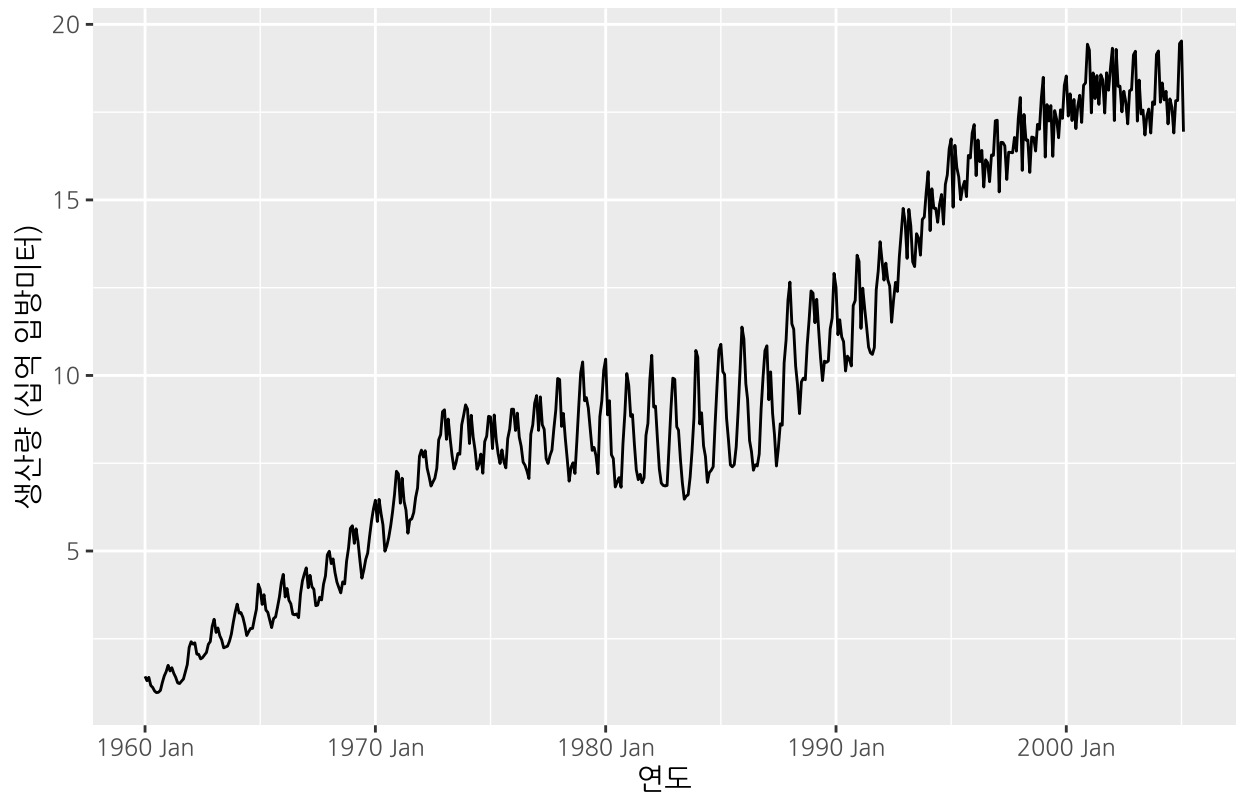
Exercise 3

문제: Why is a Box-Cox transformation unhelpful for the `canadian_gas` data?

(1) 데이터 시각화

```
canadian_gas |>
  autoplot(Volume) +
  labs(
    title = " ",
    y = " ( )",
    x = " "
  )
```

캐나다 월별 가스 생산량



(2) Box-Cox 변환 적용

```
lambda <- canadian_gas |>
  features(Volume, features = guerrero) |>
  pull(lambda_guerrero)

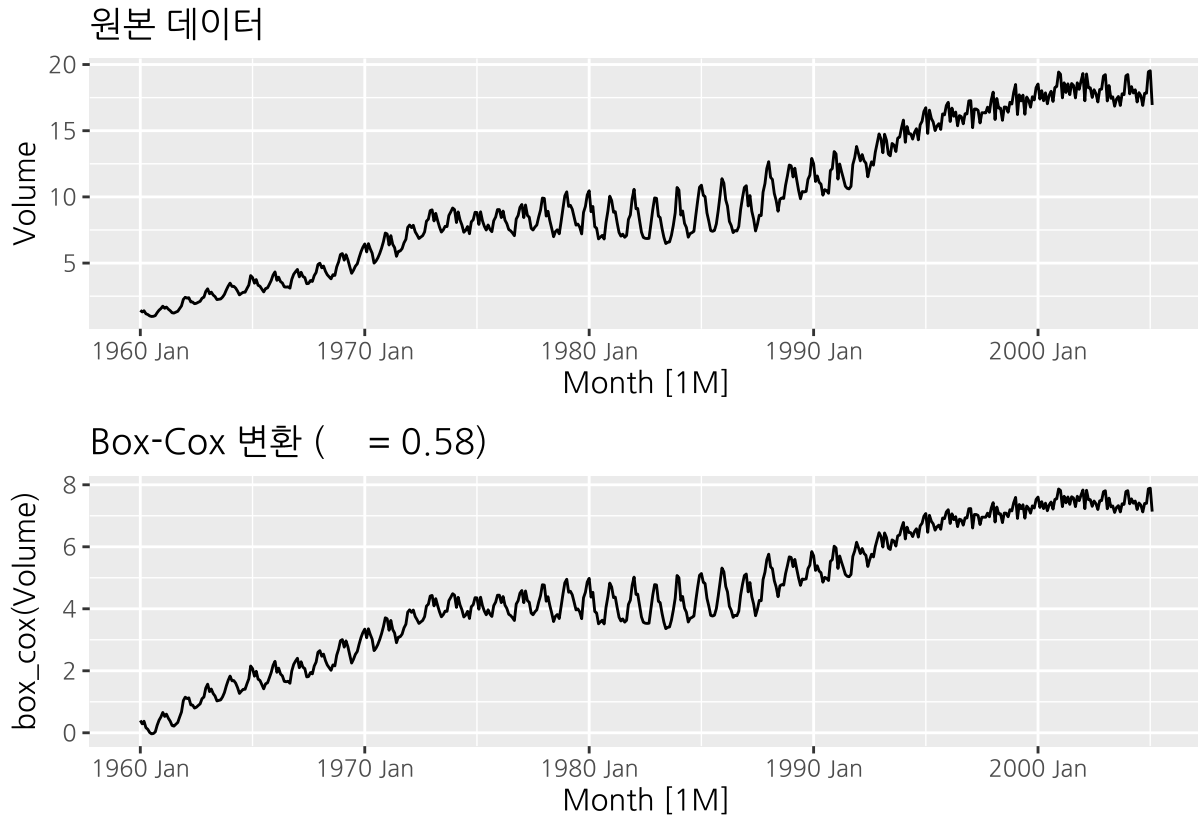
cat("    ( ):", round(lambda, 4), "\n")
```

```
##    ( ): 0.5768
```

(3) 원본 vs 변환 비교

```
p1 <- canadian_gas |>
  autoplot(Volume) +
  labs(title = "    ", y = "Volume")

p2 <- canadian_gas |>
  autoplot(box_cox(Volume, lambda)) +
  labs(
    title = paste0("Box-Cox    ( = ", round(lambda, 2), ")"),
    y = "box_cox(Volume)"
```



(4) 해석

Box-Cox 변환이 `canadian_gas` 데이터에 유용하지 않은 이유:

- Box-Cox 변환은 분산이 데이터의 수준(level)에 비례하여 증가할 때 효과적입니다.
- 그러나 `canadian_gas`는 계절성의 크기 자체가 시간에 따라 변화하는 특성을 가집니다.
 - 1960년대: 계절적 변동이 작음
 - 1970-1980년대: 계절적 변동이 크게 증가
 - 이후: 다시 감소하는 패턴
- 이처럼 분산의 변화가 데이터 수준이 아닌 시간 자체에 의해 결정되는 경우, Box-Cox 변환으로는 분산을 안정화할 수 없습니다. 위 비교 플롯에서도 변환 후에도 여전히 중간 구간에서 계절 변동이 크게 나타남을 확인할 수 있습니다.
- 따라서 이 데이터에는 변화하는 계절성을 허용하는 STL 분해가 더 적합합니다.

Exercise 5

문제: For the following series, find an appropriate Box-Cox transformation in order to stabilise the variance.

- Tobacco from `aus_production`
 - Economy class passengers between Melbourne and Sydney from `ansett`
 - Pedestrian counts at Southern Cross Station from `pedestrian`
-

(1) Tobacco from `aus_production`

```
lambda_tobacco <- aus_production |>
  features(Tobacco, features = guerrero) |>
  pull(lambda_guerrero)

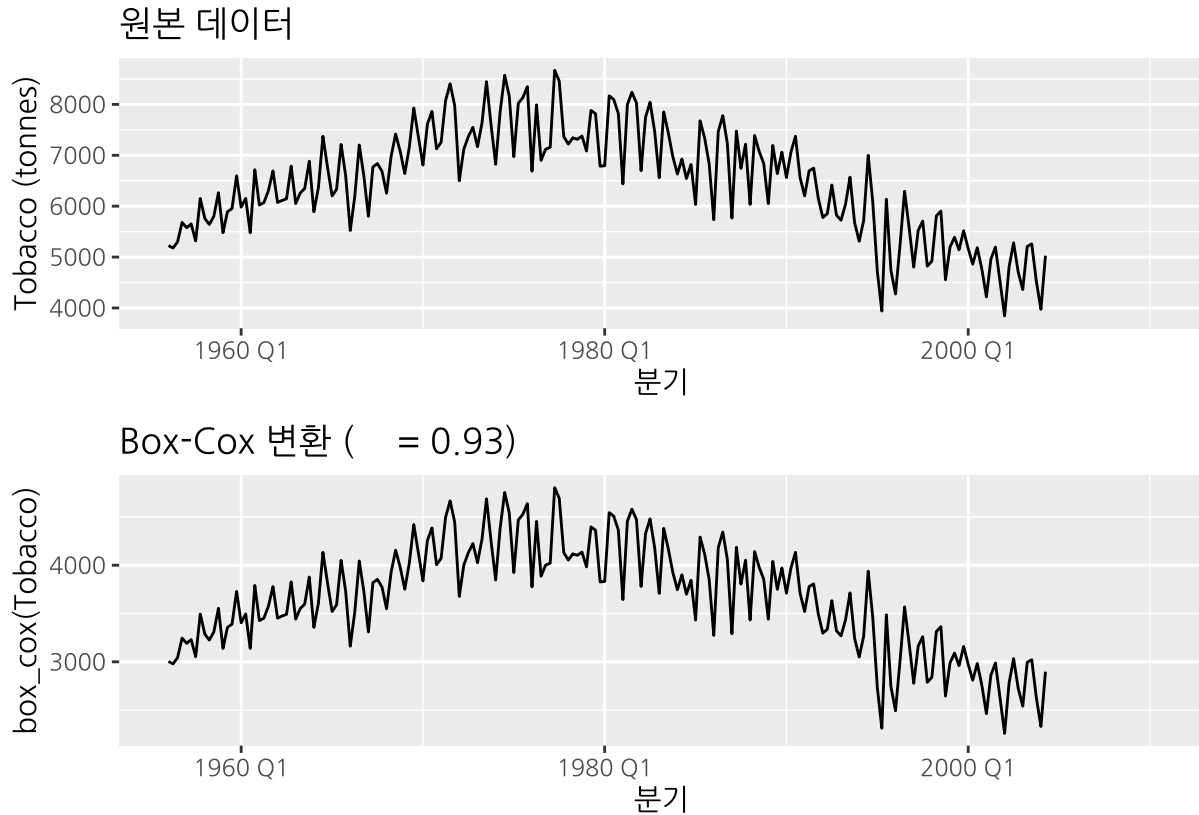
cat("Tobacco      :", round(lambda_tobacco, 4), "\n")
```

```
## Tobacco      : 0.9265
```

```
p1 <- aus_production |>
  autoplot(Tobacco) +
  labs(title = "      ", y = "Tobacco (tonnes)", x = " ")

p2 <- aus_production |>
  autoplot(box_cox(Tobacco, lambda_tobacco)) +
  labs(
    title = paste0("Box-Cox    ( = ", round(lambda_tobacco, 2), ")"),
    y = "box_cox(Tobacco)", x = " "
  )

p1 / p2
```



해석: 원본 데이터에서 시간이 지날수록 계절 변동의 폭이 증가하는 경향이 있으나, Box-Cox 변환 후 분산이 전반적으로 안정화됩니다.

(2) Economy class passengers (Melbourne $\leftrightarrow$ Sydney) from `ansett`

```
ansett_mel_sydney <- ansett |>
  filter(Airports == "MEL-SYD", Class == "Economy")

lambda_ansett <- ansett_mel_sydney |>
  features(Passengers, features = guerrero) |>
  pull(lambda_guerrero)

cat("Ansett (Economy, MEL-SYD) : ", round(lambda_ansett, 4), "\n")
```

```
## Ansett (Economy, MEL-SYD) : 1.9999
```

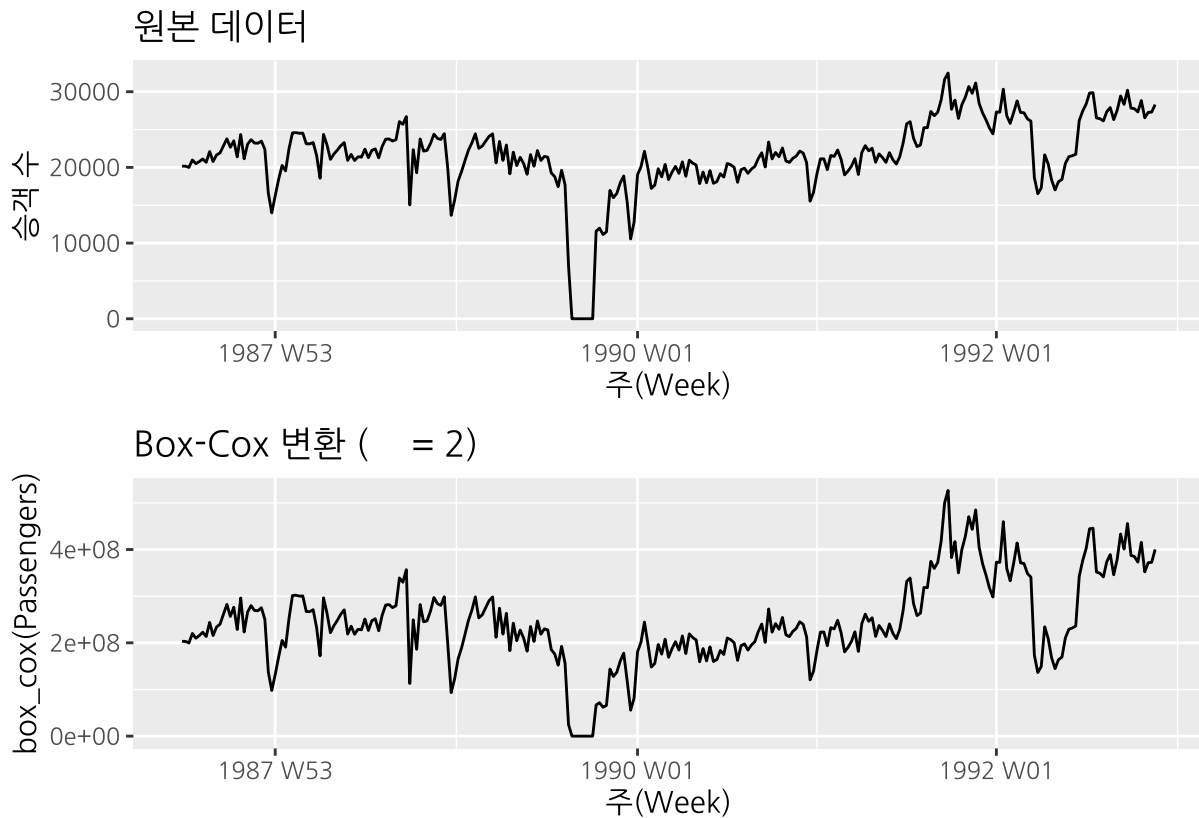
```
p1 <- ansett_mel_sydney |>
  autoplot(Passengers) +
  labs(title = " ", y = " ", x = "(Week)")

p2 <- ansett_mel_sydney |>
```



```
autoplot(box_cox(Passengers, lambda_ansett)) +
  labs(
    title = paste0("Box-Cox ( = ", round(lambda_ansett, 2), ")"),
    y = "box_cox(Passengers)", x = "(Week)"
  )
```

p1 / p2



해석: 변환 후 전반적인 분산이 안정화됩니다. 다만 1989년 항공사 파업으로 인한 급격한 변동은 Box-Cox 변환만으로는 완전히 해결되지 않으며, 이는 이상치 처리가 별도로 필요함을 시사합니다.

(3) Pedestrian counts at Southern Cross Station from `pedestrian`

```
pedestrian_scs <- pedestrian |>
  filter(Sensor == "Southern Cross Station", Count > 0) # 0 (log )

lambda_ped <- pedestrian_scs |>
  features(Count, features = guerrero) |>
  pull(lambda_guerrero)

cat("Pedestrian (Southern Cross Station) : ", round(lambda_ped, 4), "\n")
```

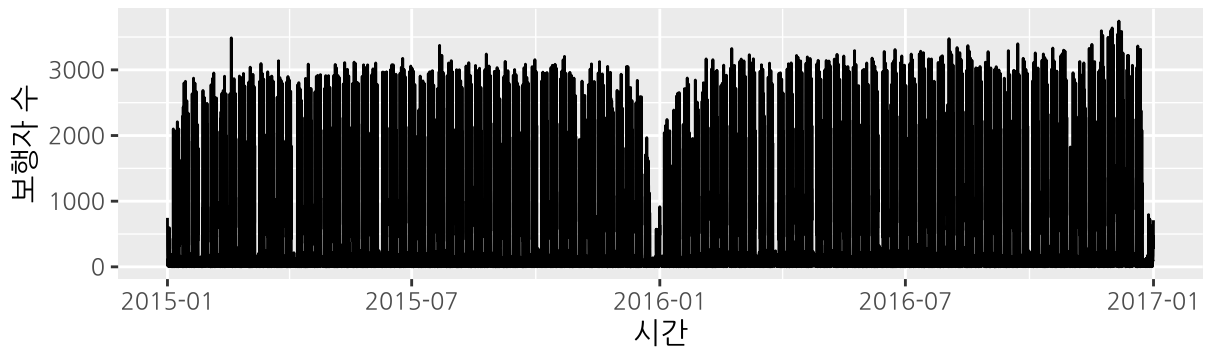
```
## Pedestrian (Southern Cross Station) : -0.166
```

```
p1 <- pedestrian_scs |>
  autoplot(Count) +
  labs(title = " ", y = " ", x = " ")

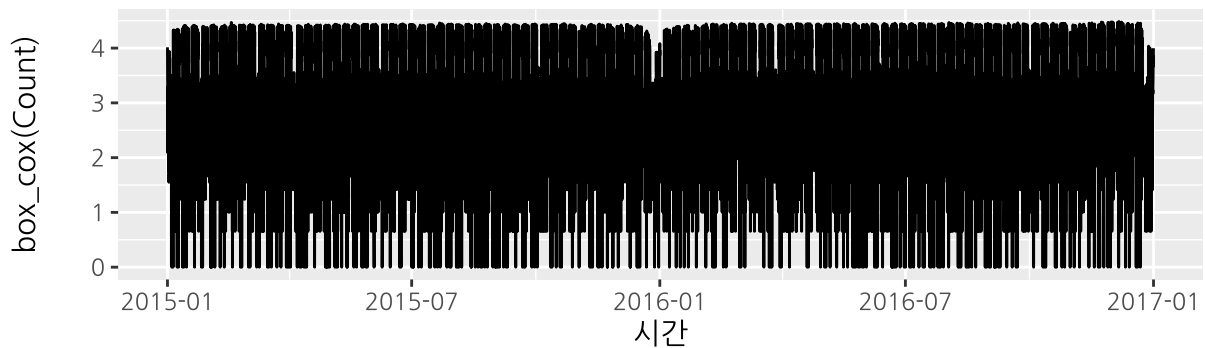
p2 <- pedestrian_scs |>
  autoplot(box_cox(Count, lambda_ped)) +
  labs(
    title = paste0("Box-Cox ( = ", round(lambda_ped, 2), ")"),
    y = "box_cox(Count)", x = " "
  )

p1 / p2
```

원본 데이터



Box-Cox 변환 ($\lambda = -0.17$)



해석: 보행자 수는 주중/주말, 출퇴근 시간대에 따라 분산이 크게 달라집니다. Count = 0인 시점은 변환 시 $-\infty$ 가 발생할 수 있어 제거 후 적용했습니다. 변환 후 분산 패턴이 어느 정도 안정화됩니다.

요약

```
tibble(
  = c(
    "Tobacco (aus_production)",
    "Economy Passengers (ansett, MEL-SYD)",
    "Pedestrian (Southern Cross Station)"
  ),
  ` ` = round(c(lambda_tobacco, lambda_ansett, lambda_ped), 4)
) |>
knitr::kable(caption = "Box-Cox ")
```

Table 1: Box-Cox 최적 λ 요약

데이터	최적 λ
Tobacco (aus_production)	0.9265
Economy Passengers (ansett, MEL-SYD)	1.9999
Pedestrian (Southern Cross Station)	-0.1660

Exercise 7

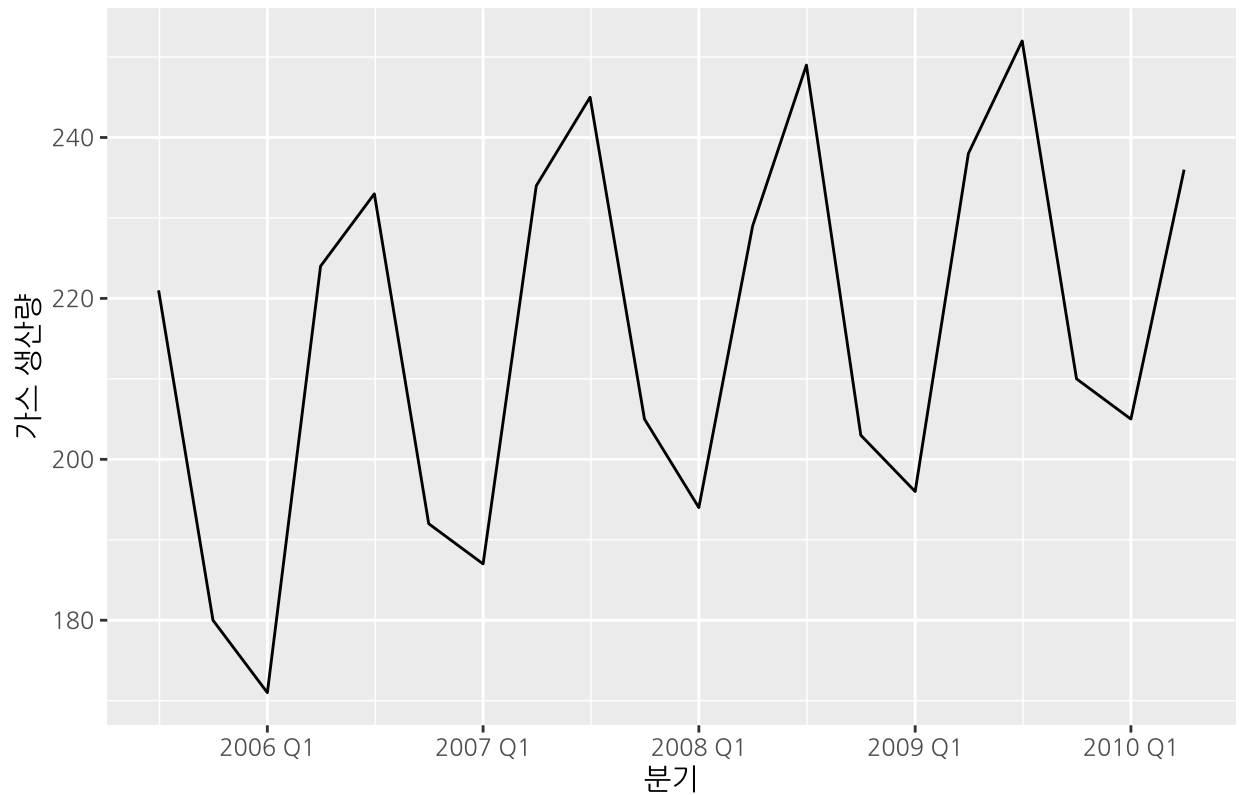
문제: Consider the last five years of the Gas data from aus_production.

```
gas <- tail(aus_production, 5 * 4) |> select(Gas)
```

(a) 시계열 플롯 — 계절성 및 추세-순환 확인

```
gas |>
autoplot(Gas) +
labs(
  title = " ( 5, 20 )",
  y = " ",
  x = " "
)
```

호주 가스 생산량 (최근 5년, 20분기)



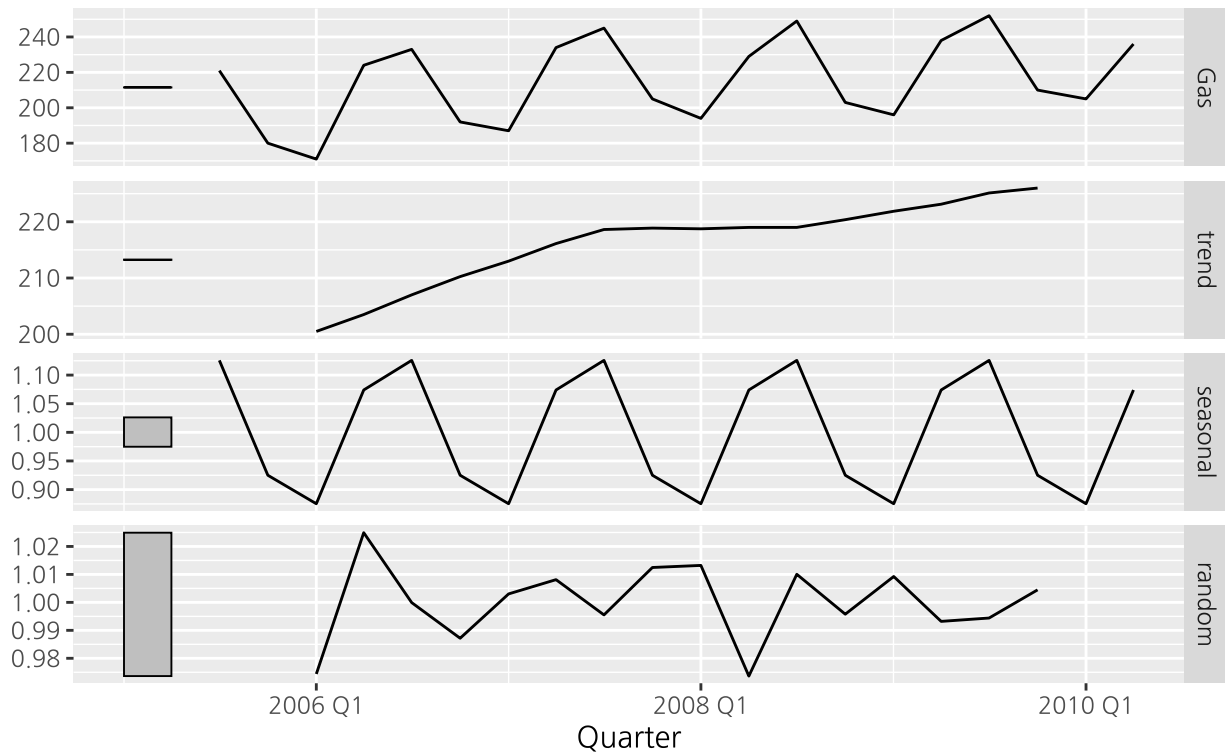
해석: 플롯에서 뚜렷한 계절적 변동(매년 반복되는 패턴)과 상승 추세가 관찰됩니다. 특정 분기에 생산량이 높아지는 패턴이 반복되며, 전반적인 수준도 시간이 지남에 따라 증가합니다.

(b) 승법적 고전 분해 (Classical Decomposition, multiplicative)

```
gas |>
  model(
    classical_decomposition(Gas, type = "multiplicative")
  ) |>
  components() |>
  autoplot() +
  labs(title = "      -      ")
```

호주 가스 생산량 — 고전적 승법 분해

Gas = trend * seasonal * random



(c) 분해 결과가 (a)의 시각적 해석을 지지하는가?

```
dcmp_gas <- gas |>
  model(
    classical_decomposition(Gas, type = "multiplicative")
  ) |>
  components()
```

dcmp_gas

```
## # A dable: 20 x 7 [1Q]
## # Key:      .model [1]
## # :        Gas = trend * seasonal * random
##   .model      Quarter  Gas trend seasonal random season_adjust
##   <chr>        <qtr> <dbl> <dbl>    <dbl> <dbl>
## 1 "classical_decomposition(G~ 2005 Q3  221   NA      1.13   NA
## 2 "classical_decomposition(G~ 2005 Q4  180   NA      0.925  NA
## 3 "classical_decomposition(G~ 2006 Q1  171  200.    0.875  0.974
## 4 "classical_decomposition(G~ 2006 Q2  224  204.    1.07   1.02
## 5 "classical_decomposition(G~ 2006 Q3  233  207.    1.13   1.000
## 6 "classical_decomposition(G~ 2006 Q4  192  210.    0.925  0.987
```

## 7	"classical_decomposition(G~ 2007 Q1	187	213	0.875	1.00	214.
## 8	"classical_decomposition(G~ 2007 Q2	234	216.	1.07	1.01	218.
## 9	"classical_decomposition(G~ 2007 Q3	245	219.	1.13	0.996	218.
## 10	"classical_decomposition(G~ 2007 Q4	205	219.	0.925	1.01	222.
## 11	"classical_decomposition(G~ 2008 Q1	194	219.	0.875	1.01	222.
## 12	"classical_decomposition(G~ 2008 Q2	229	219	1.07	0.974	213.
## 13	"classical_decomposition(G~ 2008 Q3	249	219	1.13	1.01	221.
## 14	"classical_decomposition(G~ 2008 Q4	203	220.	0.925	0.996	219.
## 15	"classical_decomposition(G~ 2009 Q1	196	222.	0.875	1.01	224.
## 16	"classical_decomposition(G~ 2009 Q2	238	223.	1.07	0.993	222.
## 17	"classical_decomposition(G~ 2009 Q3	252	225.	1.13	0.994	224.
## 18	"classical_decomposition(G~ 2009 Q4	210	226	0.925	1.00	227.
## 19	"classical_decomposition(G~ 2010 Q1	205	NA	0.875	NA	234.
## 20	"classical_decomposition(G~ 2010 Q2	236	NA	1.07	NA	220.

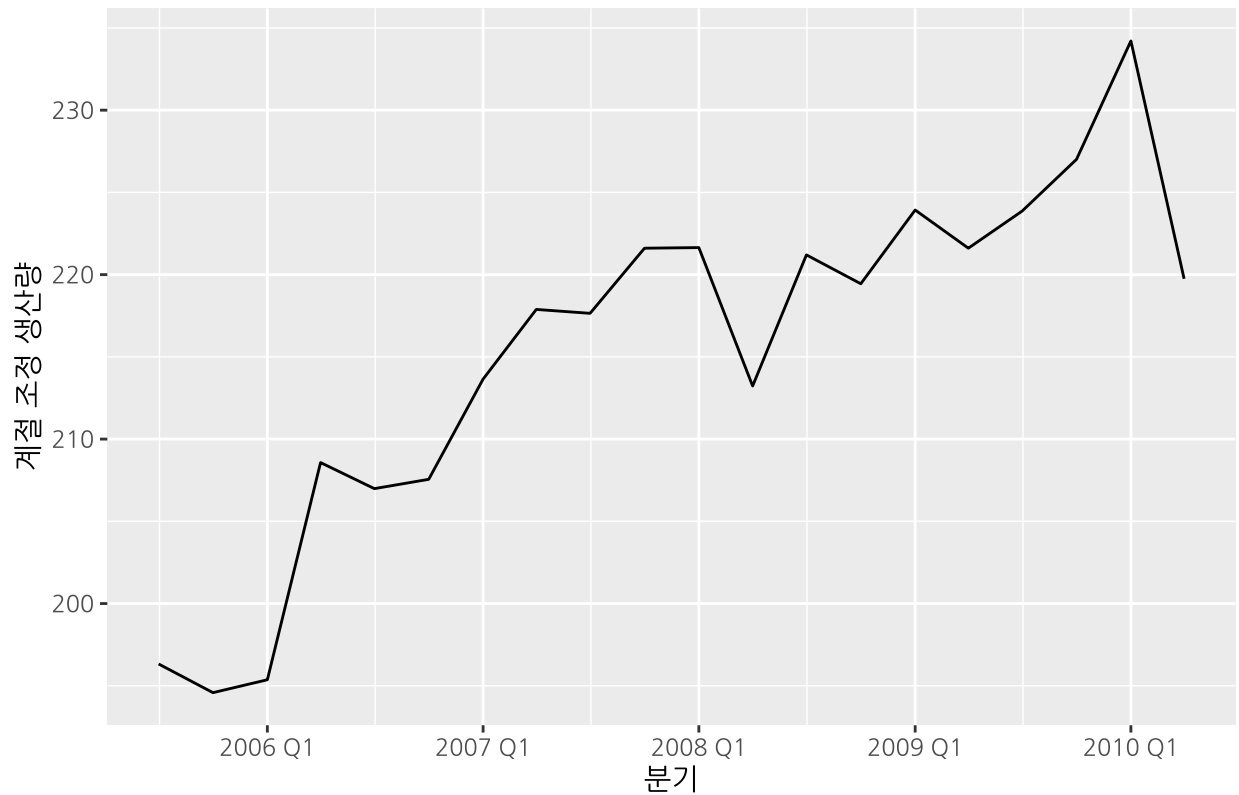
해석:

- 추세(trend) 성분이 전반적인 상승 추세를 명확히 보여줘 (a)의 해석을 지지합니다.
- 계절(seasonal) 성분은 분기별로 반복되는 일정한 패턴을 나타냅니다.
- 잔차(remainder) 성분은 1 근처의 값으로 뚜렷한 패턴이 없어, 승법 분해가 데이터를 잘 설명하고 있음을 나타냅니다.

(d) 계절 조정 데이터 계산 및 플롯

```
dcmp_gas |>
  as_tsibble() |>
  autoplot(season_adjust) +
  labs(
    title = " ",
    y = " ",
    x = " "
  )
```

계절 조정된 가스 생산량



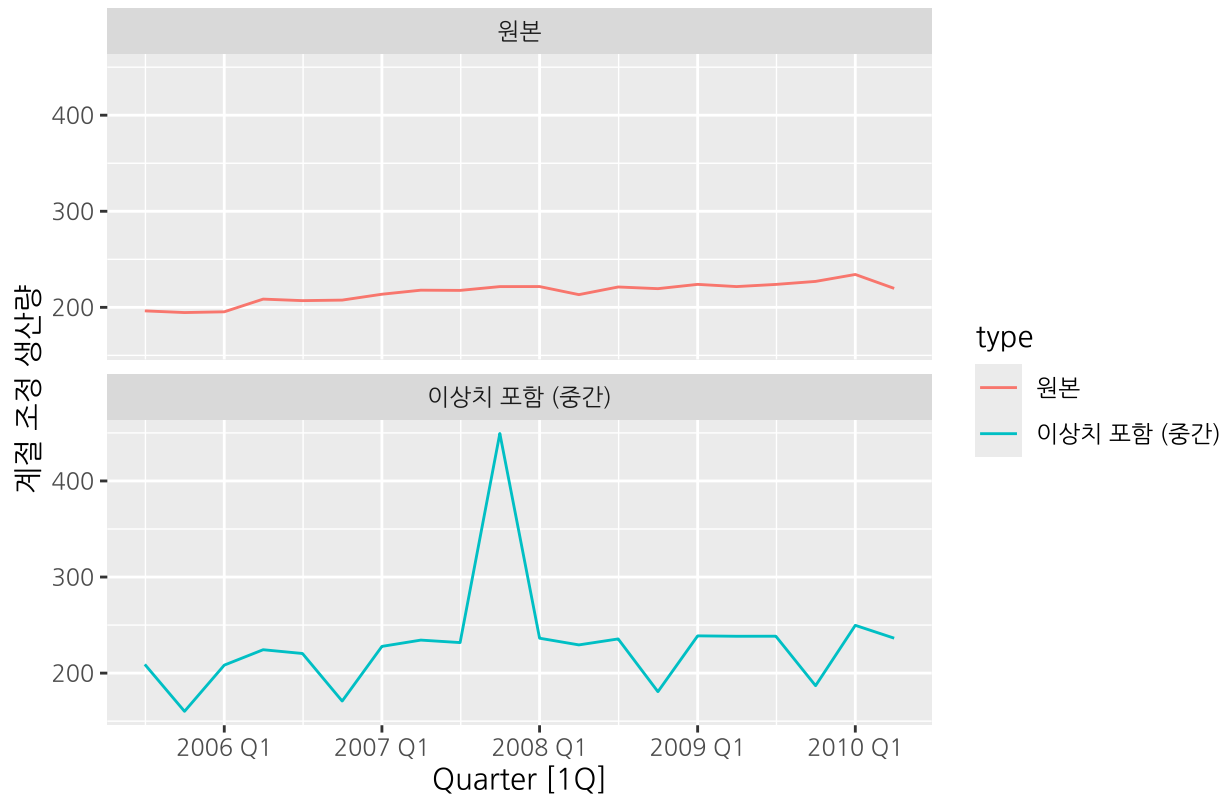
(e) 이상치(outlier) 추가 후 재계산 — 중간 지점

```
gas_outlier_mid <- gas |>
  mutate(Gas = if_else(row_number() == 10L, Gas + 300, Gas))

dcmp_mid <- gas_outlier_mid |>
  model(classical_decomposition(Gas, type = "multiplicative")) |>
  components()

bind_rows(
  dcmp_gas |> as_tibble() |> mutate(type = " "),
  dcmp_mid |> as_tibble() |> mutate(type = " ( )")
) |>
  as_tsibble(index = Quarter, key = type) |>
  autoplot(season_adjust) +
  facet_wrap(~type, ncol = 1) +
  labs(
    title = " ( vs )",
    y = " "
  )
)
```

계절 조정 데이터 비교 (원본 vs 이상치 중간 삽입)



해석: 이상치가 추가되면 해당 시점의 계절 조정 값이 크게 튀어오릅니다. 또한 이동평균으로 계산되는 추세 성분의 특성상, 이상치의 영향이 주변 시점으로 퍼지는(spreading) 효과가 나타나 인근 여러 시점의 계절 조정 값도 함께 왜곡됩니다.

(f) 이상치 위치(끝 vs 중간)에 따른 차이

```
gas_outlier_end <- gas |>
  mutate(Gas = if_else(row_number() == (n() - 1L), Gas + 300, Gas))

dcmp_end <- gas_outlier_end |>
  model(classical_decomposition(Gas, type = "multiplicative")) |>
  components()

bind_rows(
  dcmp_gas |> as_tibble() |> mutate(type = " "),
  dcmp_mid |> as_tibble() |> mutate(type = " : "),
  dcmp_end |> as_tibble() |> mutate(type = " : ")
) |>
  as_tsibble(index = Quarter, key = type) |>
  autoplot(season_adjust) +
  facet_wrap(~type, ncol = 1) +
  labs(
```

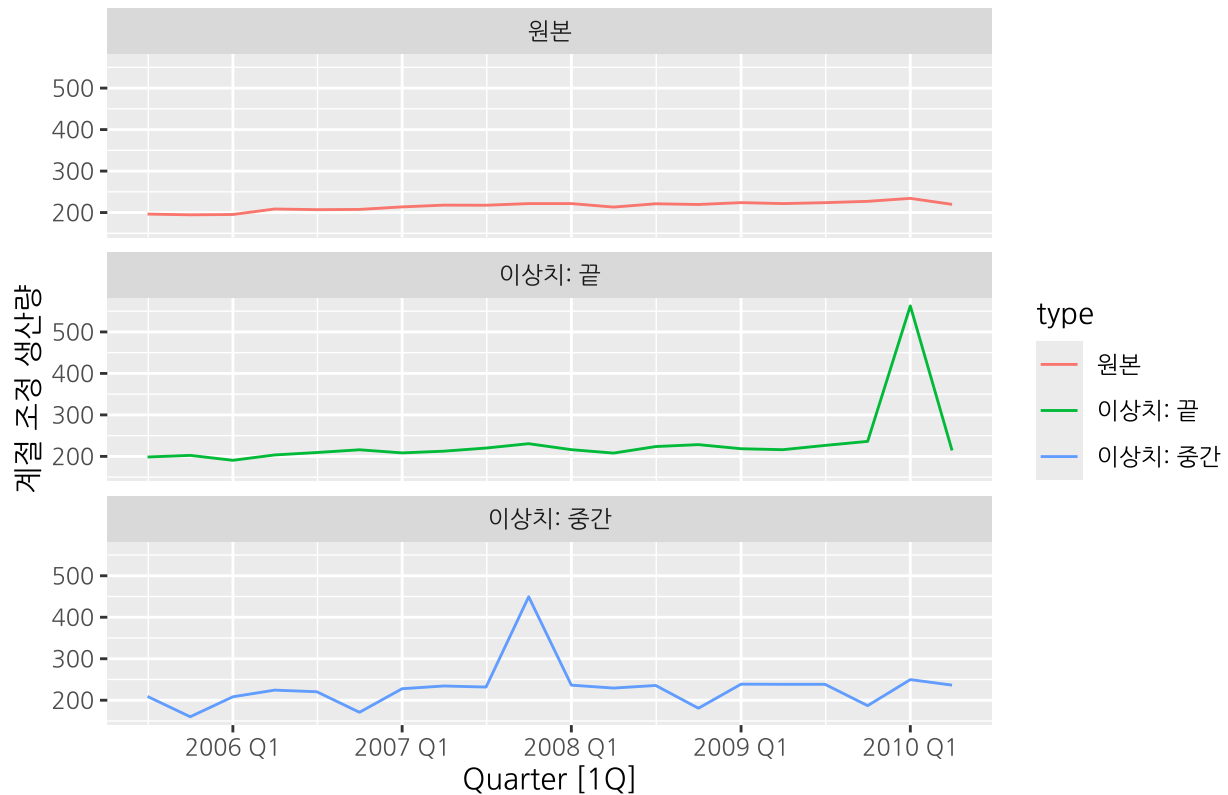


```

title = " ",
y = " "
)

```

이상치 위치에 따른 계절 조정 데이터 비교



해석:

- 중간에 이상치가 있을 때: 이동평균 윈도우가 이상치를 양쪽 방향으로 모두 포함하므로, 영향이 앞뒤 여러 시점으로 분산되어 넓은 범위에서 추세값이 왜곡됩니다.
- 끝에 이상치가 있을 때: 시계열 끝부분에서는 이동평균 계산이 불가능해 추세값이 NA로 처리됩니다. 이상치가 이동평균 윈도우에 한쪽 방향으로만 포함되므로 spreading 효과가 중간보다 상대적으로 제한됩니다. 그러나 계절 조정 값 자체에는 해당 시점에 이상치가 그대로 나타납니다.
- 어느 위치든 이상치는 계절 조정 데이터에 두드러지게 나타나므로, 이상치에 강건한 STL(robust = TRUE) 분해 방법이 더 적합합니다.

Exercise 9

문제: Figures 3.19 and 3.20 show the result of decomposing the number of persons in the civilian labour force in Australia each month from February 1978 to August 1995.

- Figure 3.19: 전체 STL 분해 결과 (원본 데이터, 추세, 계절, 잔차)
- Figure 3.20: 계절 성분 상세 (월별 패널)

본 문제는 교재에 제시된 Figure 3.19와 3.20을 직접 읽고 해석하는 문제입니다.

(a) 분해 결과 설명 (3-5문장)

1. 추세(trend) 성분은 Figure 3.19에서 약 7,000에서 9,000천 명으로 꾸준한 상승 추세를 보이며, 원본 데이터(value)의 범위와 거의 동일한 스케일을 공유합니다. 이는 전체 변동의 대부분이 장기 추세에 의해 설명됨을 의미합니다.
 2. 계절(season_year) 성분의 축 범위는 약 $-100 \sim +100$ 으로, 원본 데이터의 수준(7,000-9,000천 명)에 비해 매우 작습니다(약 $\pm 1\%$ 수준). 이는 계절적 변동이 전체 노동력에 미치는 영향이 상대적으로 미미함을 나타냅니다.
 3. Figure 3.20의 월별 계절 성분을 보면, 1월에 노동력이 가장 크게 감소하고 3월부터 회복되는 패턴이 반복됩니다. 이는 호주의 여름 방학(1월)과 연휴로 인한 일시적 고용 감소를 반영하며, 계절 패턴은 전체 기간에 걸쳐 비교적 안정적으로 유지됩니다.
 4. 잔차(remainder) 성분의 범위는 약 $-400 \sim +100$ 으로, 추세와 계절로 설명되지 않는 불규칙 변동이 존재합니다. 특히 1991-1992년 경기침체 시기에 두드러진 음의 잔차(-300 이하)가 집중적으로 나타납니다.
 5. 호주의 민간 노동력 시계열 데이터는 인구 증가와 경제 성장의 영향을 받아 장기 추세가 전체 변동을 지배합니다. 반면 계절성은 비교적 미미하며 일정한 패턴을 유지합니다. 또한 잔차를 통해 1991-1992년 경기 침체와 같은 외부 충격이 고용에 일시적인 큰 영향을 미쳤음을 확인할 수 있습니다.
-

(b) 1991/1992년 경기침체가 추정된 성분에서 보이는가?

네, 1991/1992년 경기침체는 추정된 성분들에서 확인할 수 있습니다.

- 추세 성분: Figure 3.19의 trend 패널을 보면 1991년 초부터 상승세가 꺾이고 약 1년간 하락 또는 정체하는 모습이 나타납니다. 경기침체로 인한 고용 감소가 추세에 반영된 것입니다.
- 잔차 성분: Figure 3.19의 remainder 패널에서 1991-1992년 구간에 $-300 \sim -400$ 수준의 크고 지속적인 음의 잔차가 집중적으로 나타납니다. 이는 경기침체의 갑작스럽고 강한 영향이 추세 성분만으로는 완전히 흡수되지 못했음을 나타냅니다.
- 계절 성분: 경기침체의 영향이 거의 나타나지 않으며, 이는 경기침체가 계절적 패턴 자체를 변화시키지 않았음을 의미합니다.